

Сопроводительная записка

по дисциплине «Поддержка и тестирование программных модулей»

на тему: Практическая работа №2

Специальность 09.02.04 «Информационные системы и программирование»

Исполнитель

_____/Т.А.Торган /

Группа ИПо-32

«26» Сентября 2026 г.

1. Постановка задачи	2
2. Создание проекта	2
2.1. Создание пустого проекта	2
2.2. Создание библиотеки классов	2
2.4. Создание тестов	3
2.6. Добавление ссылок	4
3. Разработка библиотеки классов	4
4. Разработка интерфейса WPF для калькулятора	7
5. Разработка логики приложения	9
6. Модульные тесты	11
7. Сборка и запуск	13
8. Заключение	13

1. Постановка задачи

Разработать приложение "Калькулятор" на WPF с поддержкой операций:

Сложение (+)

Вычитание (-)

Умножение (*)

Деление (/)

Возведение в степень (^)

Скобки для изменения приоритета

Логика вычислений должна быть вынесена в отдельную библиотеку классов.

Написать модульные тесты.

2. Создание проекта

2.1. Создание пустого проекта

Visual Studio - Создать проект - Новое решение

Имя: CalculatorTorgan

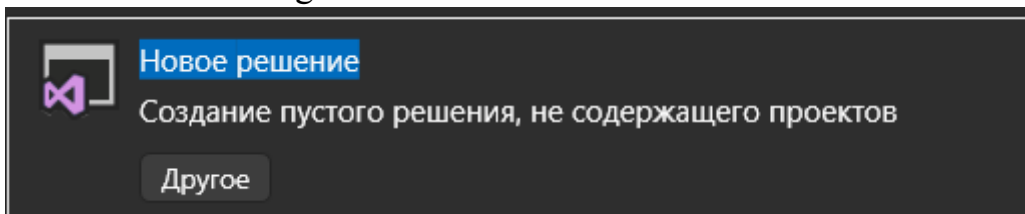


Рисунок 1 – Создание пустого решения

2.2. Создание библиотеки классов

Пкм на решение "CalculatorTorgan" - Добавить - создать проект - Библиотека классов (.NET Framework)

Имя: CalculatorLibrary

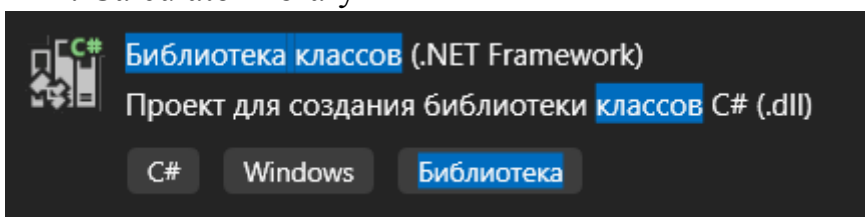


Рисунок 2 – Создание библиотеки классов CalculatorLibrary

2.3. Создание WPF приложения

Пкм на решение “CalculatorTorgan” - Добавить - создать проект - Приложение WPF

Имя: CalculatorApp

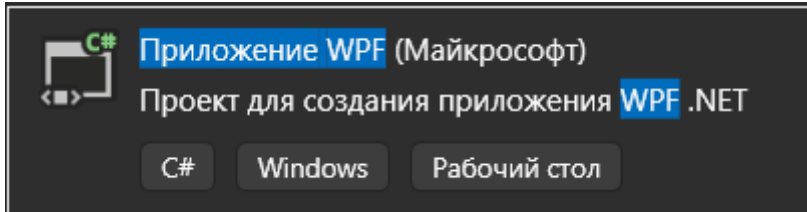


Рисунок 3 – Создание WPF-приложения CalculatorApp

2.4. Создание тестов

Пкм на решение “CalculatorTorgan” - Добавить - создать проект - Проект Модульного теста WPF

Имя: CalculatorTests

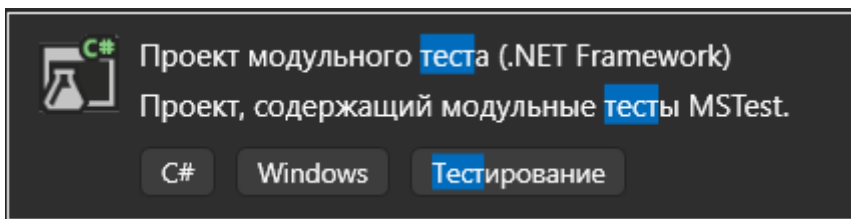


Рисунок 4 – Создание проекта модульных тестов CalculatorTests

2.5 Итог создания в обозревателе

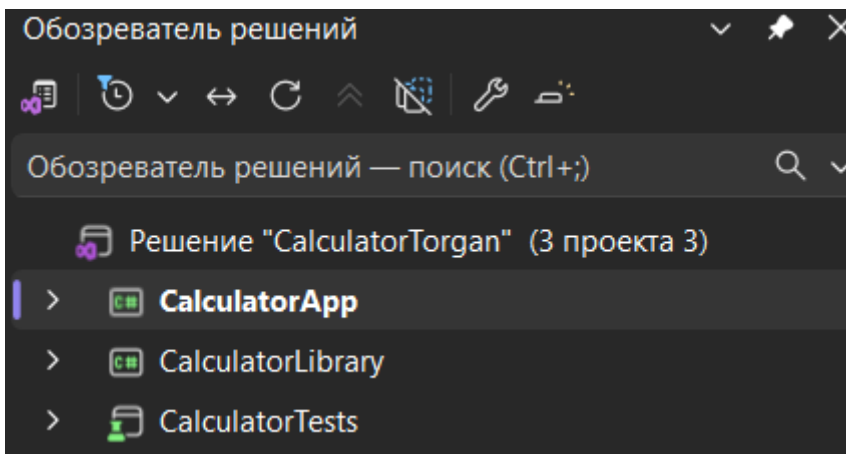


Рисунок 5 – Структура решения в обозревателе решений

2.6. Добавление ссылок

В проектах CalculatorApp и CalculatorTests добавить ссылку на CalculatorLibrary:

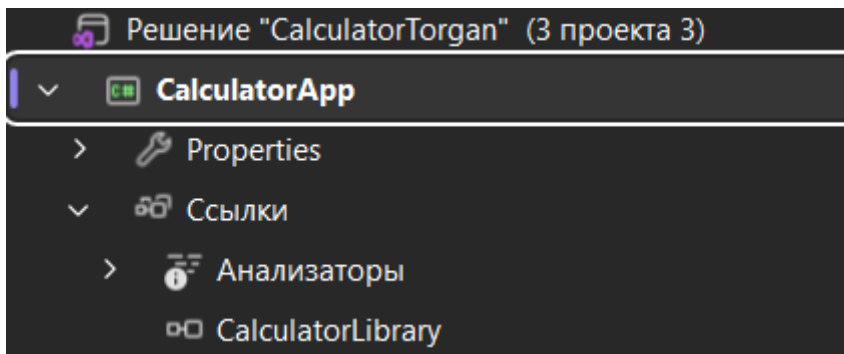


Рисунок 6 - Добавление ссылки в проекте CalculatorApp

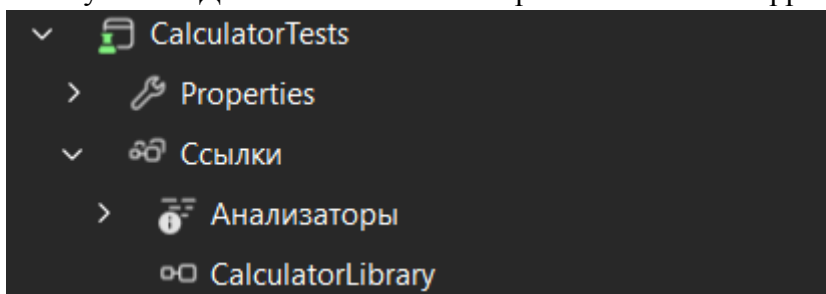


Рисунок 7 – Добавление ссылки в проекте CalculatorTests

3. Разработка библиотеки классов

Первым этапом разработки стала реализация библиотеки классов, содержащей основную логику вычислений. Это было сделано для разделения ответственности: интерфейс отвечает только за ввод и вывод данных, а все математические операции вынесены в отдельный компонент. Библиотека содержит класс CalculatorEngine, в котором реализованы метод Evaluate для вычисления выражения целиком и метод Calculate для выполнения одной операции над двумя числами. В методе Evaluate реализован парсер выражений методом рекурсивного спуска, который поддерживает приоритет операций, скобки и унарный минус (например, $-2^2 = 4$). Также добавлена проверка деления на ноль - при попытке выполнить такую операцию выбрасывается исключение, которое затем обрабатывается в приложении.

```

1  using System;
2  using System.Collections.Generic;
3
4  namespace CalculatorLibrary
5  {
6      Ссылка: 4
7      public class CalculatorEngine
8      {
9          Ссылка: 1
10         public double Evaluate(string expression)
11         {
12             if (string.IsNullOrEmpty(expression))
13                 throw new ArgumentException("пустое выражение");
14
15             expression = expression.Replace(" ", "");
16             int pos = 0;
17             double result = ParseExpression(expression, ref pos);
18
19             if (pos < expression.Length)
20                 throw new InvalidOperationException("ошибка в выражении");
21
22             return result;
23         }
24         Ссылка: 7
25         public double Calculate(double a, double b, string operation)
26         {
27             switch (operation)
28             {
29                 case "+": return a + b;
30                 case "-": return a - b;
31                 case "*": return a * b;
32                 case "/":
33                     if (b == 0)
34                         throw new DivideByZeroException("деление на ноль невозможно!");
35                     return a / b;
36                 case "^": return Math.Pow(a, b);
37                 default:
38                     throw new InvalidOperationException("неизвестная операция!");
39             }
40         }
41     }
42 }

```

Рисунок 8 – Код класса CalculatorEngine (начало)

```

35         throw new InvalidOperationException("неизвестная операция!");
36     }
37 }
38
39 // сложение и вычитание
40 Ссылка: 2
41 private double ParseExpression(string s, ref int pos)
42 {
43     double left = ParseTerm(s, ref pos);
44
45     while (pos < s.Length && (s[pos] == '+' || s[pos] == '-'))
46     {
47         char op = s[pos];
48         pos++;
49         double right = ParseTerm(s, ref pos);
50         left = op == '+' ? left + right : left - right;
51     }
52
53     return left;
54 }
55
56 // умножение и деление
57 Ссылка: 2
58 private double ParseTerm(string s, ref int pos)
59 {
60     double left = ParsePower(s, ref pos);
61
62     while (pos < s.Length && (s[pos] == '*' || s[pos] == '/'))
63     {
64         char op = s[pos];
65         pos++;
66         double right = ParsePower(s, ref pos);
67
68         if (op == '/' && right == 0)
69             throw new DivideByZeroException("деление на ноль невозможно!");
70
71         left = op == '*' ? left * right : left / right;
72     }
73
74     return left;
75 }

```

Рисунок 9 – Код класса CalculatorEngine (продолжение)

```

70         }
71     }
72     return left;
73 }
74
75 // возведение в степень
76 Ссылка: 3
77 private double ParsePower(string s, ref int pos)
78 {
79     double left = ParseUnary(s, ref pos);
80
81     if (pos < s.Length && s[pos] == '^')
82     {
83         pos++;
84         double right = ParsePower(s, ref pos); // рекурсия
85         left = Math.Pow(left, right);
86     }
87     return left;
88 }
89
90 // минус и плюс, скобки, числа
91 Ссылка: 3
92 private double ParseUnary(string s, ref int pos)
93 {
94     if (pos < s.Length && s[pos] == '-')
95     {
96         pos++;
97         return -ParseUnary(s, ref pos);
98     }
99
100     if (pos < s.Length && s[pos] == '+')
101     {
102         pos++;
103         return ParseUnary(s, ref pos);
104     }

```

Рисунок 9 – Код класса CalculatorEngine (продолжение)

```

104     return ParseAtom(s, ref pos);
105 }
106
107 // скобки и числа
108 Ссылка: 1
109 private double ParseAtom(string s, ref int pos)
110 {
111     if (pos >= s.Length)
112         throw new InvalidOperationException("конец выражения");
113
114     // скобка
115     if (s[pos] == '(')
116     {
117         pos++;
118         double value = ParseExpression(s, ref pos);
119
120         if (pos >= s.Length || s[pos] != ')')
121             throw new InvalidOperationException("не закрыта скобка");
122
123         pos++;
124         return value;
125     }
126
127     // число
128     int start = pos;
129
130     while (pos < s.Length && (char.IsDigit(s[pos]) || s[pos] == '.'))
131         pos++;
132
133     if (start == pos)
134         throw new InvalidOperationException($"ошибка на позиции {pos}");
135
136     string number = s.Substring(start, pos - start);
137     return double.Parse(number, System.Globalization.CultureInfo.InvariantCulture);
138 }
139 }

```

Рисунок 9 – Код класса CalculatorEngine (конец)

4. Разработка интерфейса WPF для калькулятора

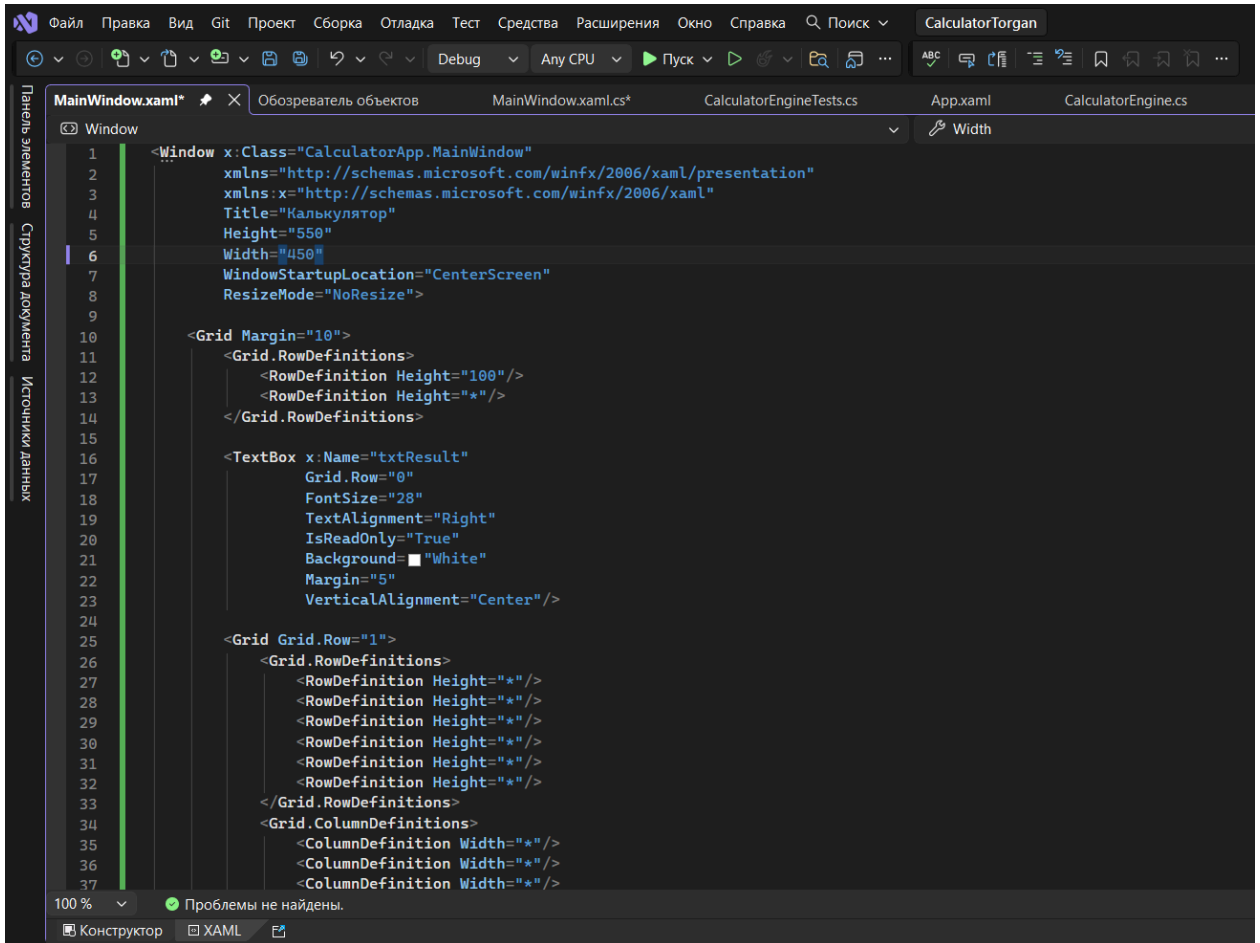


Рисунок 10 – Разметка MainWindow.xaml (начало)

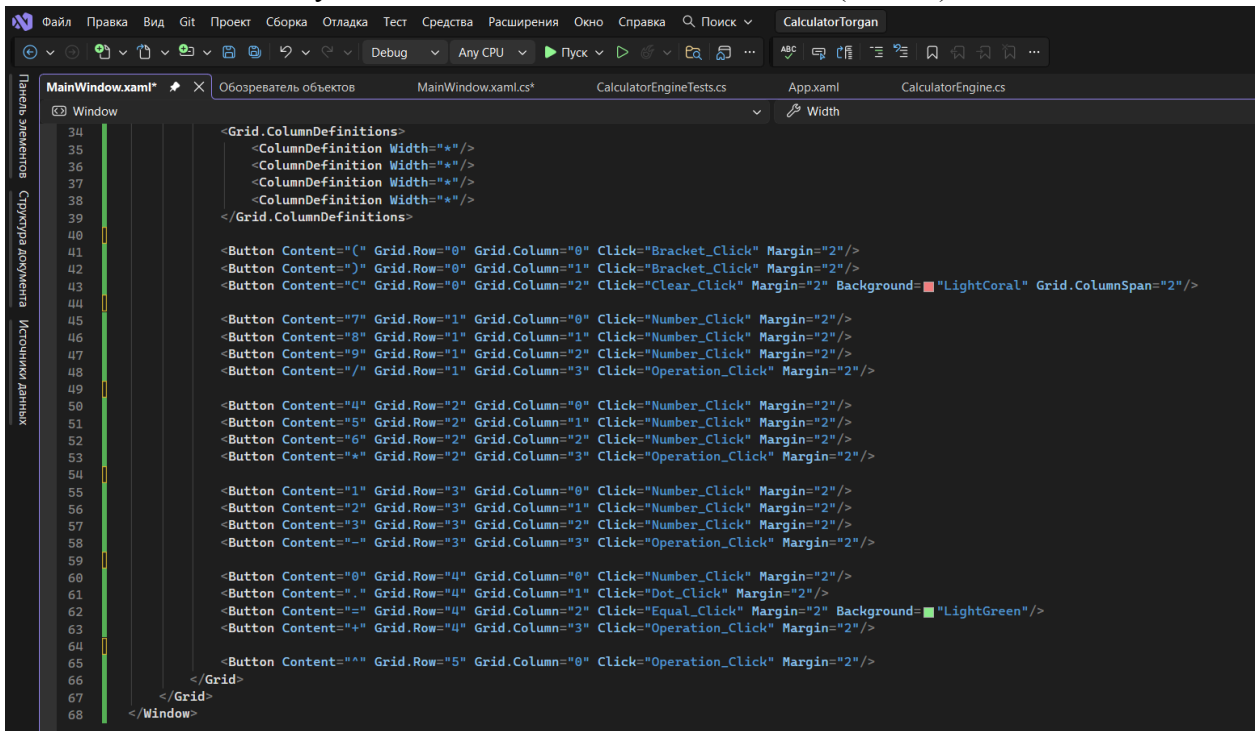


Рисунок 11 – Разметка MainWindow.xaml (кнопки)

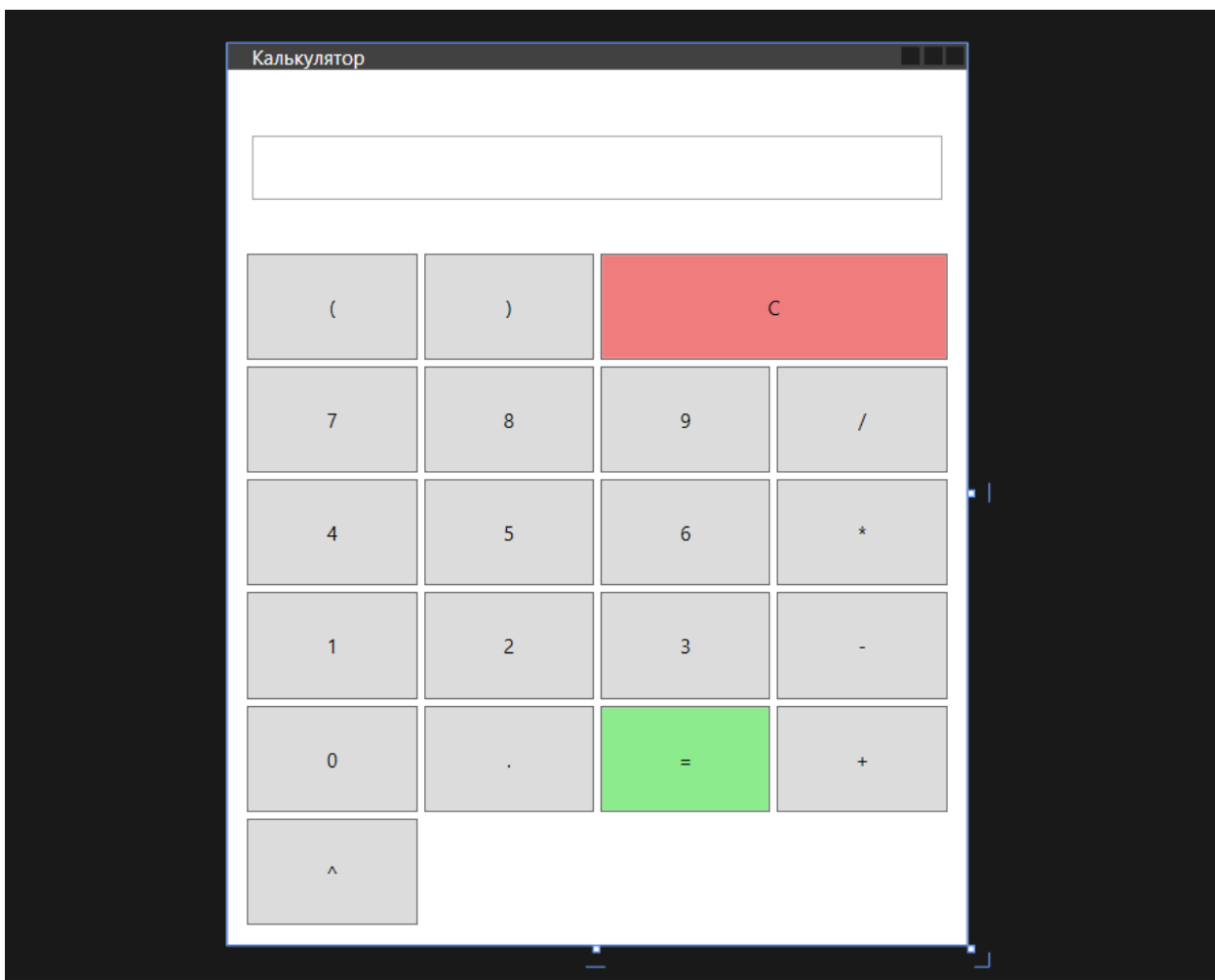


Рисунок 12 – Разработанный интерфейс WPF

Интерфейс приложения создавался с помощью языка разметки XAML. Основными элементами стали:

Поле TextBox с именем txtResult для отображения вводимого выражения и результата вычислений. Поле сделано доступным только для чтения, чтобы пользователь не мог редактировать его напрямую, а ввод осуществлялся только через кнопки.

Сетка кнопок, организованная с помощью элемента Grid, который разбит на 6 строк и 4 столбца. Каждая кнопка имеет обработчик события Click, который определяет, какое действие необходимо выполнить (добавить цифру, операцию или скобку).

Кнопки операций вынесены в отдельные обработчики, что позволило удобно управлять поведением приложения. Кнопка очистки (C) сбрасывает все введенные данные и устанавливает в поле значение "0".

Для удобства пользователя кнопки сгруппированы логически: в верхнем ряду расположены скобки и кнопка очистки, затем идут цифры и основные операции, а в нижнем ряду — кнопка возведения в степень.

5. Разработка логики приложения

Логика приложения была написана на языке C# в файле `MainWindow.xaml.cs`. Основная задача WPF-приложения — собирать строку выражения и передавать её в библиотеку `CalculatorEngine.Evaluate()`. Это соответствует требованию о разделении логики и интерфейса: вся математика выполняется в библиотеке.

Основные методы:

`Number_Click` — обрабатывает нажатие цифровых кнопок. Если в поле отображается «0», число заменяется, иначе цифра добавляется к текущему выражению.

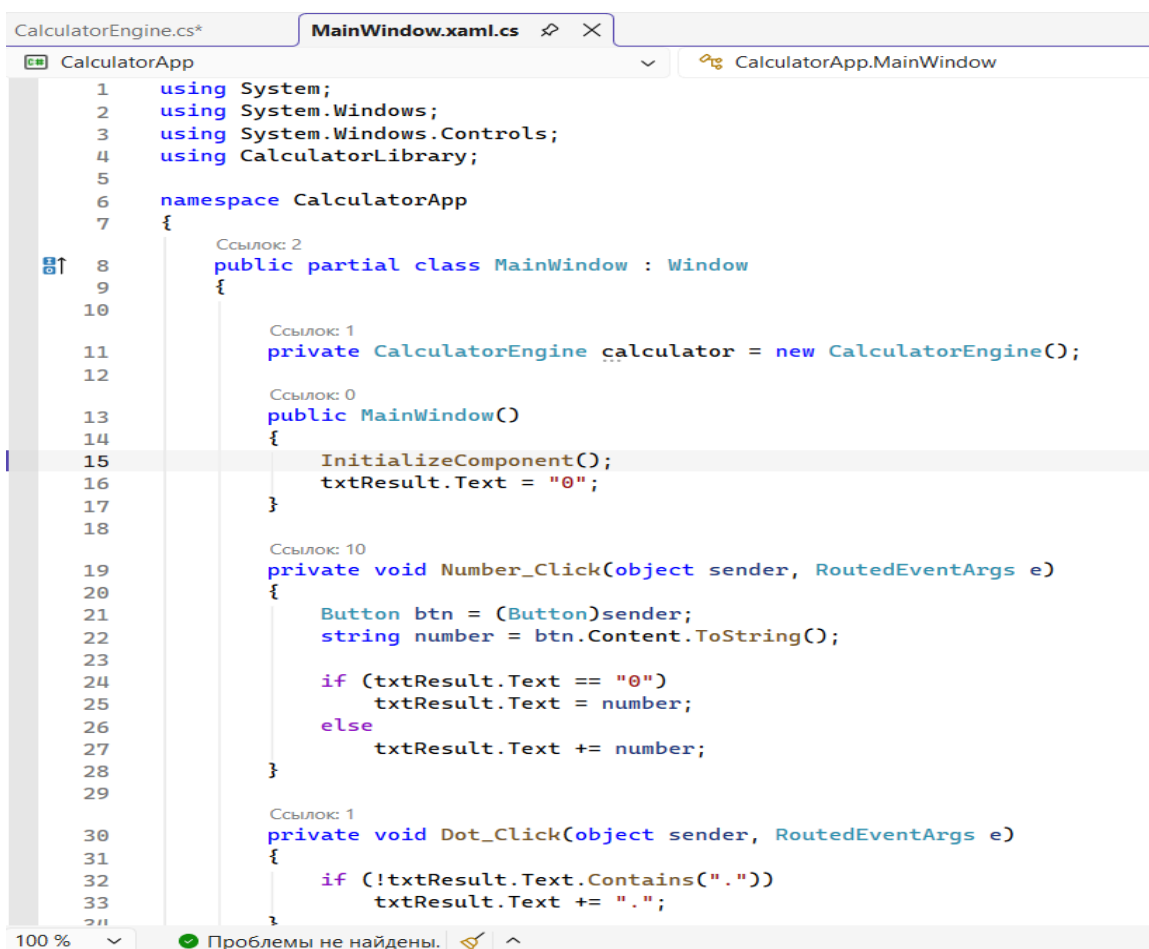
`Dot_Click` — добавляет десятичную точку.

`Operation_Click` — обрабатывает нажатие кнопок операций. Если на экране «0», он заменяется на знак операции (это позволяет вводить отрицательные числа с самого начала), иначе знак добавляется к выражению.

`Bracket_Click` — добавляет скобку к текущему выражению.

`Equal_Click` — главный метод. Получает строку выражения и передаёт её в `calculator.Evaluate()`. Результат выводится в поле. При ошибках выводится сообщение, а поле сбрасывается в «0».

`Clear_Click` — очищает поле, устанавливая значение «0».



```
1 using System;
2 using System.Windows;
3 using System.Windows.Controls;
4 using CalculatorLibrary;
5
6 namespace CalculatorApp
7 {
8     Ссылка: 2
9     public partial class MainWindow : Window
10     {
11         Ссылка: 1
12         private CalculatorEngine calculator = new CalculatorEngine();
13
14         Ссылка: 0
15         public MainWindow()
16         {
17             InitializeComponent();
18             txtResult.Text = "0";
19         }
20
21         Ссылка: 10
22         private void Number_Click(object sender, RoutedEventArgs e)
23         {
24             Button btn = (Button)sender;
25             string number = btn.Content.ToString();
26
27             if (txtResult.Text == "0")
28                 txtResult.Text = number;
29             else
30                 txtResult.Text += number;
31         }
32
33         Ссылка: 1
34         private void Dot_Click(object sender, RoutedEventArgs e)
35         {
36             if (!txtResult.Text.Contains("."))
37                 txtResult.Text += ".";
38         }
39
40         Ссылка: 1
41         private void Equal_Click(object sender, RoutedEventArgs e)
42         {
43             string result = calculator.Evaluate(txtResult.Text);
44             txtResult.Text = result;
45         }
46     }
47 }
```

Рисунок 13 – Код `MainWindow.xaml.cs` (начало)

```

34     }
35
36     private void Bracket_Click(object sender, RoutedEventArgs e)
37     {
38         Button btn = (Button)sender;
39         string bracket = btn.Content.ToString();
40
41         if (txtResult.Text == "0")
42             txtResult.Text = bracket;
43         else
44             txtResult.Text += bracket;
45     }
46
47     private void Operation_Click(object sender, RoutedEventArgs e)
48     {
49         Button btn = (Button)sender;
50         string operation = btn.Content.ToString();
51
52         if (txtResult.Text == "0")
53         {
54             txtResult.Text = operation;
55         }
56         else
57         {
58             txtResult.Text += $" {operation} ";
59         }
60     }
61
62     private void Equal_Click(object sender, RoutedEventArgs e)
63     {
64         try
65         {
66             // вызов библиотеки
67             double result = calculator.Evaluate(txtResult.Text);

```

Рисунок 14 – Код MainWindow.xaml.cs (продолжение)

```

67         double result = calculator.Evaluate(txtResult.Text);
68         txtResult.Text = result.ToString(System.Globalization.CultureInfo.InvariantCulture);
69     }
70     catch (DivideByZeroException)
71     {
72         MessageBox.Show("на ноль делить нельзя!", "ошибка",
73             MessageBoxButton.OK, MessageBoxImage.Error);
74         txtResult.Text = "0";
75     }
76     catch (Exception ex)
77     {
78         MessageBox.Show($"ошибка в выражении: {ex.Message}", "ошибка",
79             MessageBoxButton.OK, MessageBoxImage.Error);
80         txtResult.Text = "0";
81     }
82 }
83
84 private void Clear_Click(object sender, RoutedEventArgs e)
85 {
86     txtResult.Text = "0";
87 }
88 }
89 }

```

Рисунок 15 – Код MainWindow.xaml.cs (конец)

6. Модульные тесты

Для проверки корректности работы библиотеки классов были разработаны модульные тесты с использованием фреймворка MSTest. Тесты размещены в отдельном проекте CalculatorTests. Каждый тест проверяет конкретную операцию или выражение:

Test_Addition — проверяет сложение ($5 + 3 = 8$)

Test_Subtraction — проверяет вычитание ($10 - 4 = 6$)

Test_Multiplication — проверяет умножение ($7 \times 6 = 42$)

Test_Division — проверяет деление ($15 / 3 = 5$)

Test_DivisionByZero — проверяет, что при делении на ноль выбрасывается исключение

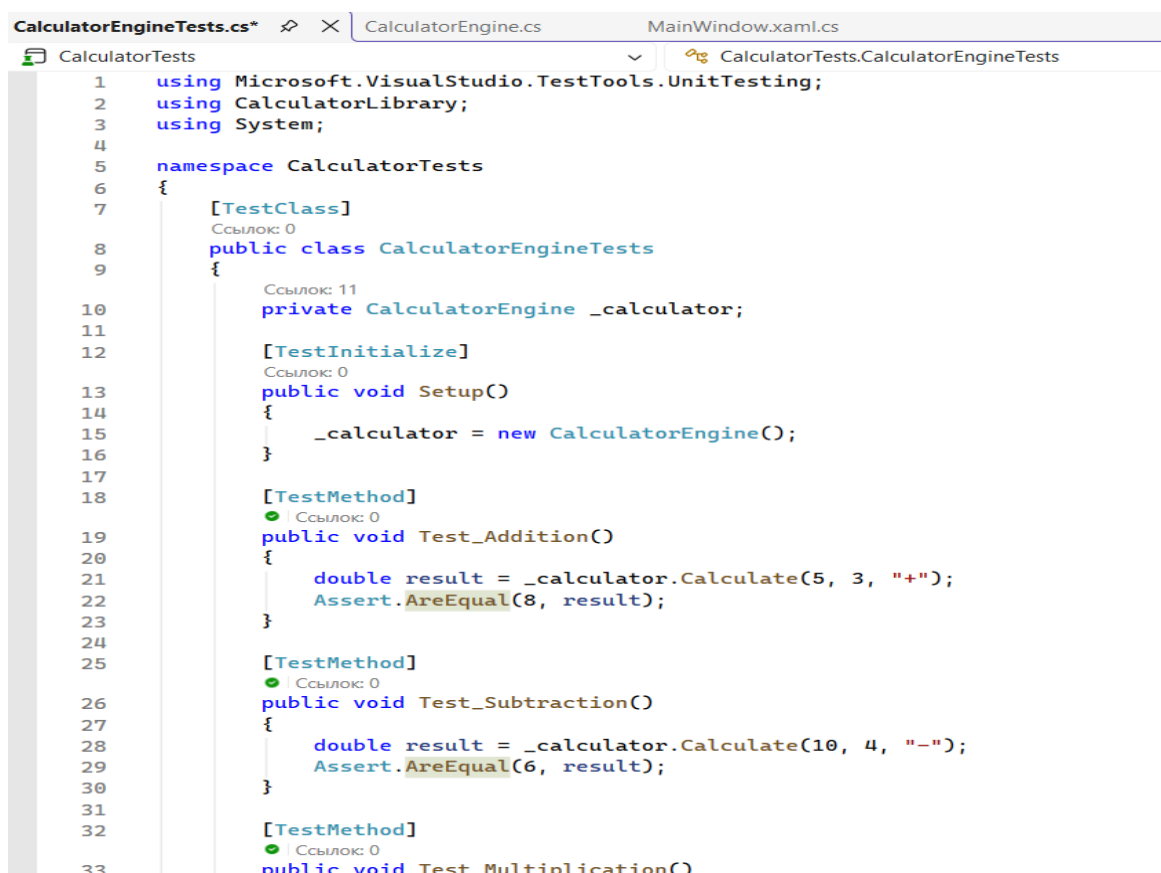
Test_Power — проверяет возведение в степень ($2^3 = 8$)

Test_UnknownOperation — проверяет, что при неизвестной операции выбрасывается исключение

Test_Evaluate_WithUnaryMinusAndBrackets — проверяет выражение $-2^2 + (1 + 4) = 9$

Test_Evaluate_WithPriority — проверяет выражение $2 + 3 * 4 = 14$

Test_Evaluate_WithBrackets — проверяет выражение $(2 + 3) * 4 = 20$



```
1  using Microsoft.VisualStudio.TestTools.UnitTesting;
2  using CalculatorLibrary;
3  using System;
4
5  namespace CalculatorTests
6  {
7      [TestClass]
8      Ссылка: 0
9      public class CalculatorEngineTests
10     {
11         Ссылка: 11
12         private CalculatorEngine _calculator;
13
14         [TestInitialize]
15         Ссылка: 0
16         public void Setup()
17         {
18             _calculator = new CalculatorEngine();
19         }
20
21         [TestMethod]
22         Ссылка: 0
23         public void Test_Addition()
24         {
25             double result = _calculator.Calculate(5, 3, "+");
26             Assert.AreEqual(8, result);
27         }
28
29         [TestMethod]
30         Ссылка: 0
31         public void Test_Subtraction()
32         {
33             double result = _calculator.Calculate(10, 4, "-");
34             Assert.AreEqual(6, result);
35         }
36
37         [TestMethod]
38         Ссылка: 0
39         public void Test_Multiplication()
```

Рисунок 16 – Код модульных тестов CalculatorEngineTests (начало)

```

32 [TestMethod]
33     Ссылка: 0
34     public void Test_Multiplication()
35     {
36         double result = _calculator.Calculate(7, 6, "*");
37         Assert.AreEqual(42, result);
38     }
39
40 [TestMethod]
41     Ссылка: 0
42     public void Test_Division()
43     {
44         double result = _calculator.Calculate(15, 3, "/");
45         Assert.AreEqual(5, result);
46     }
47
48 [TestMethod]
49     Ссылка: 0
50     public void Test_DivisionByZero()
51     {
52         _calculator.Calculate(10, 0, "/");
53     }
54
55 [TestMethod]
56     Ссылка: 0
57     public void Test_Power()
58     {
59         double result = _calculator.Calculate(2, 3, "^");
60         Assert.AreEqual(8, result);
61     }
62
63 [TestMethod]
64     Ссылка: 0
65     public void Test_UnknownOperation()

```

Рисунок 17 – Код модульных тестов CalculatorEngineTests (продолжение)

```

62     public void Test_UnknownOperation()
63     {
64         _calculator.Calculate(5, 3, "%");
65     }
66
67
68 [TestMethod]
69     Ссылка: 0
70     public void Test_Evaluate_WithUnaryMinusAndBrackets()
71     {
72         Assert.AreEqual(9, _calculator.Evaluate("-2^2+(1+4)"));
73     }
74
75 [TestMethod]
76     Ссылка: 0
77     public void Test_Evaluate_WithPriority()
78     {
79         Assert.AreEqual(14, _calculator.Evaluate("2+3*4"));
80     }
81
82 [TestMethod]
83     Ссылка: 0
84     public void Test_Evaluate_WithBrackets()
85     {
86         Assert.AreEqual(20, _calculator.Evaluate("(2+3)*4"));
87     }
88 }

```

Рисунок 18 – Код модульных тестов CalculatorEngineTests(конец)

